

File Operations

Objectives

1. C++ I/O streams.
2. Reading and writing sequential files.
3. Reading and writing Random Access Files.

File

“A file is a collection of information, usually stored on a computer’s disk. Information can be saved to files and then later reused.”

File Names

All files are assigned a name that is used for identification purposes by the operating system and the user.

Some File Names with Extension

File Name and Extension	File Contents
MYPROG.BAS	BASIC program
MENU.BAT	DOS Batch File
INSTALL.DOC	Documentation File
CRUNCH.EXE	Executable File
BOB.HTML	HTML (Hypertext Markup Language) File
3DMODEL.JAVA	Java program or applet
INVENT.OBJ	Object File
PROG1.PRJ	Borland C++ Project File
ANSI.SYS	System Device Driver
README.TXT	Text File

Why File?

(all the message or value printed with help of any output statements like cout , put are never available for future use)

1. If there is a large amount of data generated as output by a program, storing that output in file will help in easy handling /analysis of the output as user can see the whole output at any time even after complete execution of the program.
2. If we need lot of data to be inputted, user cannot keep on typing that again and again for repeated execution of program. In that case, all input data can be once written in a file and then that file can be easily used as the input file.
3. The transfer of input – data or output – data from one computer to another can be easily done by using files.

The Process of Using a File

Using a file in a program is a simple three-step process

1. The file must be opened. If the file does not yet exist, opening it means creating it.
2. Information is then saved to the file, read from the file, or both.
3. When the program is finished using the file, the file must be closed.

Stream Classes

Ifstream: provides input operations.

Contains **open()** with default input mode.

Inherits the functions

get(), **getline()**, **read()**, **seekg()** and **tellg()** function from **istream**.

Ofstream: provides output operations.

Contains **open()** with default output mode.

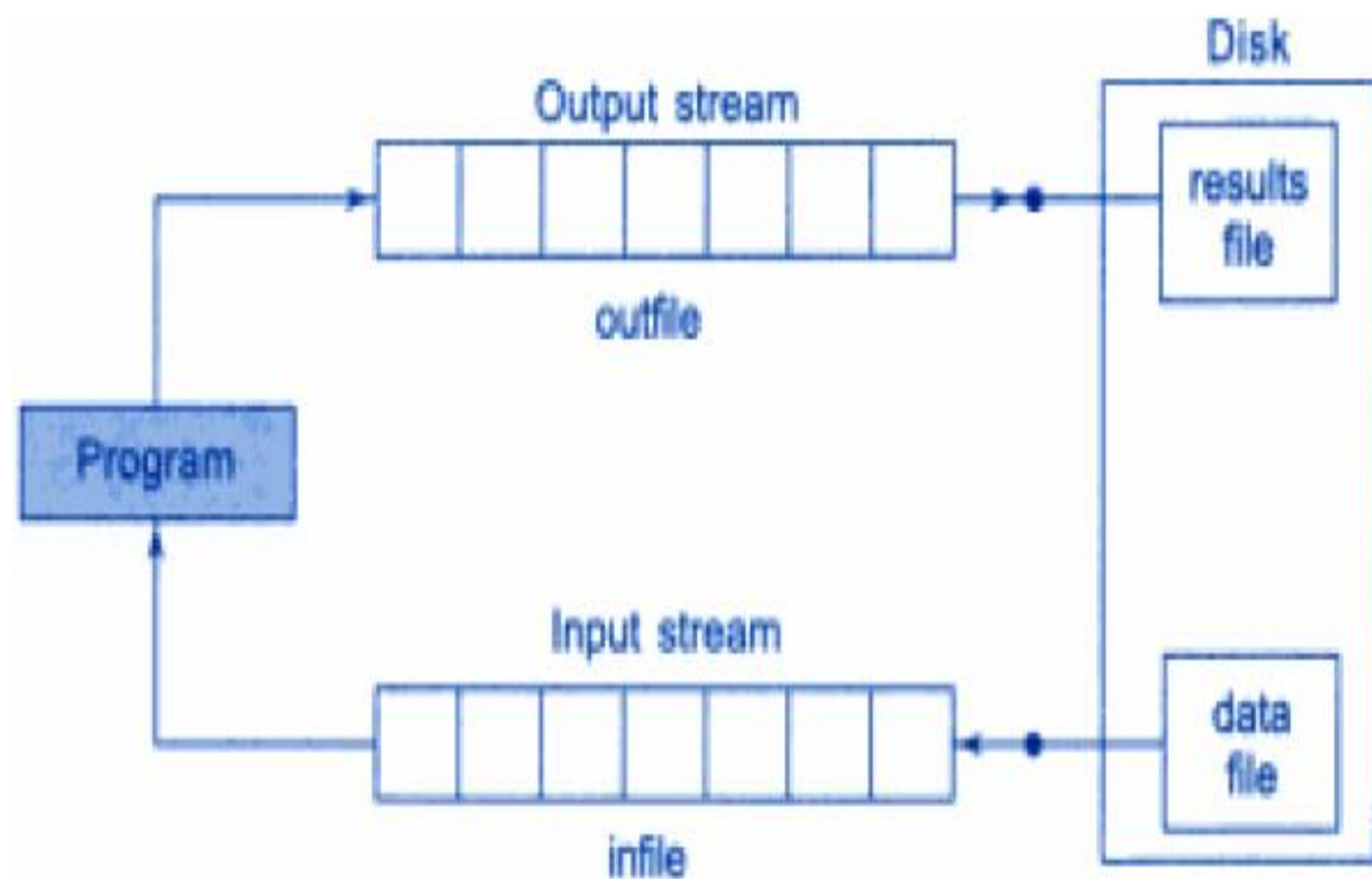
Inherits

put(), **seekp()**, **teelp()** and **write()** function from **ostream**.

Fstream : provides support for simultaneous input and output operations.

Contains **open()** with default input mode.

Inherits all the function from **istream** and **ostream** classes through **iostream**



- C++ views each files as a sequence of bytes
- Each file ends with an *end-of-file* marker.
- When a file is *opened*, an object is created and a stream is associated with the object.
- To perform file processing in C++,
the header files **<iostream>** and **<fstream>**
must be included.
- **<fstream.>** includes **<ifstream>** and **<ofstream>**

File Open Modes

1. `ios:: app` - (append) write all output to the end of file
2. `ios:: ate` - data can be written anywhere in the file
3. `ios:: binary` - read/write data in binary format
4. `ios:: in` - (input) open a file for input
5. `ios::out` - (output) open a file for output
6. `ios: trunc` -(truncate) discard the files' contents if it exists
7. `ios:nocreate` - if the file does NOT exists, the open operation fails.
8. `ios:noreplace` - if the file exists, the open operation fails

Open and Close a file

eg:-

```
ofstream outfile;           // create stream
outfile . open ("DATA1"); // connect stream to DATA1
.....
.....
outfile . Close();          //disconnect stream from DATA1

outfile . Open("DATA2"); //connect stream to DATA2
.....
.....
outfile . close();
```

The mode can combine two or more parameters using the bitwise **OR** operator (symbol |)

eg :-

```
fstream file;
```

```
file . Open(“ data . txt”, ios :: out | ios :: in);
```

Opening a File

For opening a file, we must first create a file stream and then link it to the filename.

A file can be opened in two ways:

1. Using the **constructor function** of the class.
2. Using the **member function open()** of the class.

Opening File using Constructor

A constructor is used to initialize an object while it is being created.

In the same manner, a file name is used to initialize the file stream object.

This involves the following steps:

1. Create a file stream object to manage the stream using the appropriate class, i.e. the class `ofstream` is used to create the output stream and the class `ifstream` is used to create the input stream.
2. Initialized the file object with the desired filename.

e.g:

```
Ofstream outfile("results"); //output only
```

This statement opens the file results and attaches it to the output stream outfile.

```
Ifstream infile("data"); //input only
```

Similarly, above statement declares infile as an ifstream object and attaches it to the file data for reading (input)

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;

int main()
{
    ofstream outfile("Jai.txt");
    cout<<"enter Name";
    string name;
    cin>>name;

    outfile<<name;
    outfile.close();

    ifstream infile("Jai.txt");
    infile>>name;
    cout<<"\nname= "<<name<<endl;

    system("pause");
    return 0;
}
```

Note

When a file is opened for writing only,

- a new file is created if there is no file of that name.
- if a file by that name exists already , then its contents are deleted and the file is presented as a clean file .

How to open an existing file for updating it without loosing its original contents?

Opening Files Using open()

1. The function `open()` can be used to open multiple files that use the same stream object.
2. If we want to open file sequentially, we may create a single stream object and use it to open each file in turn.

```
file-stream-class stream-object;  
stream-object.open("filename");
```

Working with Multiple Files

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;
```

```
int main()
{
    ofstream outfile;

    outfile.open("Next.txt");

    outfile<<"Tashkent "<<endl;
    outfile<<"Samarkhand"<<endl;
    outfile<<"Bukhara"<<endl;
    outfile.close();

    outfile.open("First.txt");

    outfile<<"Tashkent- Frist"<<endl;
    outfile<<"Samarkhand- first"<<endl;
    outfile<<"Bukhara- first"<<endl;

    outfile.close();
```

```
    string line;
    ifstream infile;
    infile.open("First.txt");
    while(infile)
    {
        getline(infile,line);
        cout<<line<<endl;
    }
    infile.close();

    infile.open("next.txt");
    while(infile)
    {
        getline(infile,line);
        cout<<line<<endl;
    }
    infile.close();
    system("pause");
    return 0;
}
```

Detecting END-OF-FILE

- Detection of end-of-file condition necessary for preventing any further attempt to read data from the file.

while(fin)

- An ifstream object, such as fin, **returns a value of 0** if any error occurs in the file operation including the end-of-file condition.

If(fin.eof()!=0){exit(1);}

eof is a member function of IOS class.

It returns a non-zero value if the end-of-file(EOF) condition is encountered, and a zero, otherwise.

Reading from Two Files Simultaneously

```
#include<iostream>
#include<string>
#include<fstream>
using namespace std;
int main()
{
    ofstream outfile1, outfile2;
    string name;
    outfile1.open("Country.txt");
    outfile2.open("Capital.txt");
    for (int i = 0; i < 3; i++)
    {   cout << "Input Country Name: ";
        getline(cin, name);
        outfile1 << name << endl;
        cout << "Enter Country Capital: ";
        getline(cin, name);
        outfile2 << name << endl;
    }
    outfile1.close(); outfile2.close();
}
```

```
ifstream infile1, infile2;
infile1.open("Country.txt");
infile2.open("Capital.txt");
for (int i = 0; i < 3; i++)
{   if (infile1.eof()==0)
    {       getline(infile1, name);
            cout << name << "          ";   }
    if (infile2.eof() == 0)
    {       getline(infile2, name);
            cout << name << "          ";
    }
    cout << endl;
}
|   system("pause");
return 0;
}
```

Program 2

```
int main()
{
    ofstream outfile;
    outfile.open("odd.txt");
    for(int i=0;i<10;i++)
    {
        if(i%2!=0)
            outfile<<i<<endl;
    }
    outfile.close();

    outfile.open("even.txt");
    for(int i=0;i<10;i++)
    {
        if(i%2==0)
            outfile<<i<<endl;
    }
    outfile.close();

    int num;
```

```
ifstream infile;
infile.open("odd.txt");
cout<<"\n ODD NUMBERS ARE :- \n";
while (infile.eof()==0)
{
    infile >> num;
    cout<<num<<endl;
}
infile.close();
infile.open("even.txt");
cout<<"\n EVEN NUMBERS ARE :- \n";
while((infile.eof()==0)
{
    infile>>num;
    cout<<num<<endl;
}
infile.close();
system("pause");
return 0;
}
```

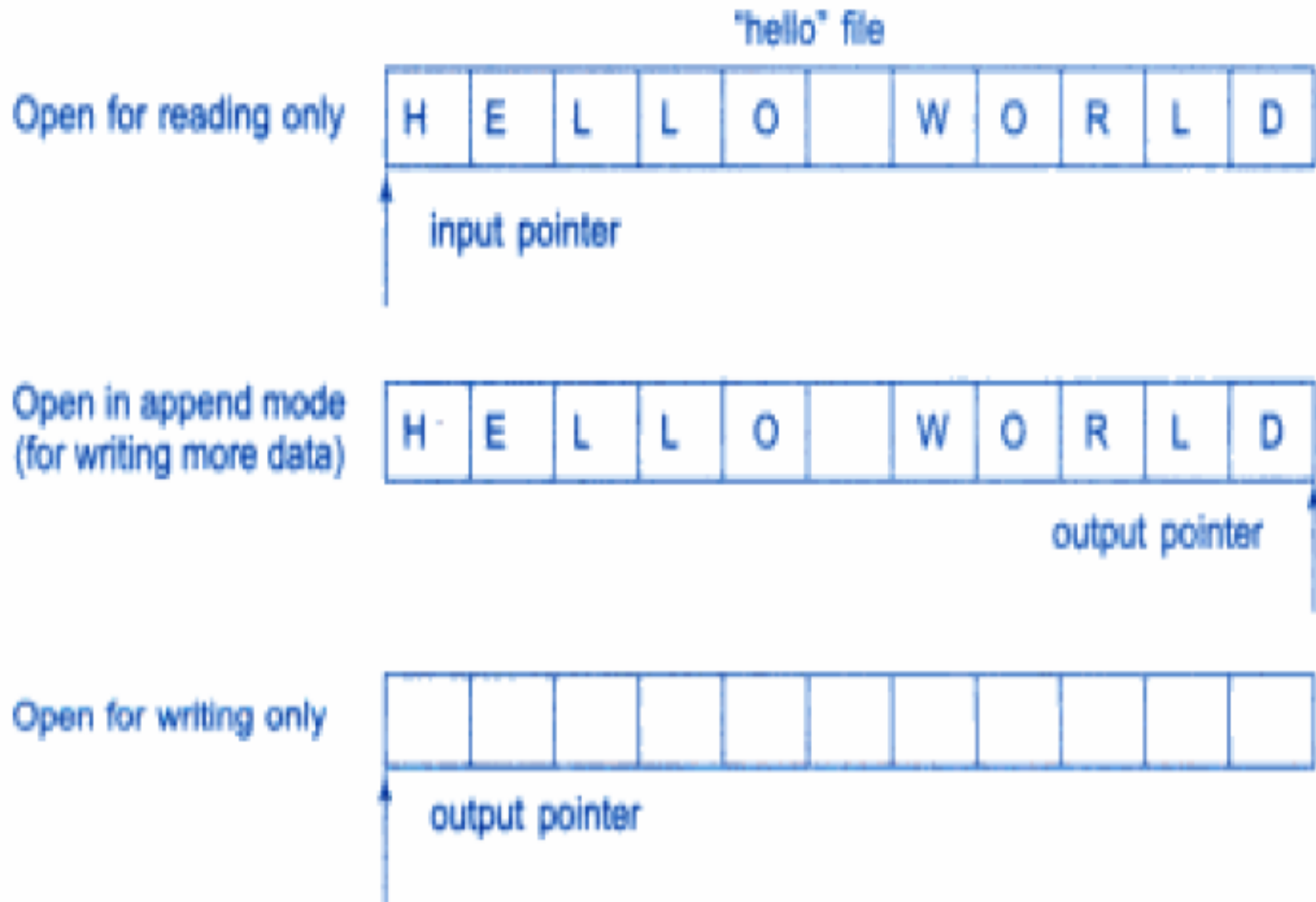
File Pointers

1. Each file have two associated pointers known as the file pointers.
2. One of them is called the input pointer (or **get pointer**) and the other is called the output pointer (or **put pointer**).
3. The input pointer is used for reading the contents of a given file location
4. The output pointer is used for writing to a given file location.

Default Actions

1. To open a file in **read-only** mode, the input pointer is automatically set at the beginning so that we can read the file from the start.
2. Similarly, when we open a file in **write-only** mode, the existing contents are deleted and output pointer is set at the beginning. This enables us to write to the file from the start .
3. If we want to open an existing file to add more data, the file is opened in '**append**' mode.

Action on File Pointers while Opening a File



Function for manipulation of file pointer

When we want to move file pointer to desired position then use these function for manage the file pointers.

seekg () = moves get pointer (input) to a specified location

seekp () = moves put pointer (output) to a specified location

tellg () = gives the current position of the get pointer

tellp () = gives the current position of the put pointer

Example,

Infile.seekg(10) ;

This statement moves the file pointer to the byte number 10.

The bytes in the a file are numbered beginning from zero.

Therefore, the pointer will be pointing to the 11th byte in the file.

Consider the following statements:

ofstream fileout;

Fileout.open(“hello”,ios::app);

Int p=fileout.tellp();

On execution of these statement, The pointer is moved to the end of the file “hello” and the value of p Will represent number of bytes in the file.

‘Seek’ functions

seekg() and seekp() can also be used with two arguments as follows:

Seekg(offset,refposition); , seekp(offset,refposition)

Offset represents the number of bytes the file pointer is to be moved from the location specified by the parameter ref position.

Reference position takes one of the following three constants defined in the ios class

ios::beg start of the file

ios:: cur current position of the pointer

ios::end end of the file

<code>fout . seekg(0, ios :: beg) --</code>	go to start
<code>fout . seekg(0, ios :: cur) --</code>	stay at current position
<code>fout . seekg(0, ios :: end) --</code>	go to the end of file
<code>fout . seekg(m, ios :: beg) --</code>	move to m+1 byte in the file
<code>fout . seekg(m, ios :: cur) --</code>	go forward by m bytes from the current position
<code>fout . seekg(-m, ios :: cur) --</code>	go backward by m bytes from the current position
<code>fout . seekg(-m, ios :: end) --</code>	go backward by m bytes from the end

put() and get() functions

The function `put()` write a single character to the associated stream.

Similarly,
the function `get()` reads a single character from the associated stream.

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;

int main() {
    std::ofstream outfile("test.txt");
    char ch;
    std::cout << "Type some text (type a dot to finish):\n";
    do {
        ch = std::cin.get();
        outfile.put(ch);
    } while (ch != '.');
    system("pause");
    return 0;
```

read() and write() function

1. The functions `read()` and `write ()` , unlike the functions `put()` and `get()` , handle the data in binary form.
2. Values are stored in the disk file in the same format in which they are stored in the internal memory.
3. An int take two bytes to store its value in the binary form ,irrespective of its size.But a 4 digit int will take 4 bytes to store it in the character form.
4. The binary format is more accurate for storing the numbers as they are stored in the exact internal representation.
5. There are no conversions while saving the data and therefore saving is much faster.

```
file.read ((char *)&V , sizeof (V));  
file.write ((char *)&V , sizeof (V));
```

These function take two arguments. The first is the address of the variable V , and the second is the length of that variable in bytes . The address of variable must be cast to type char * (i.e pointer to character type) .

To output objects of other types, we must convert the pointers to those objects to type const char*

```
#include<iostream>
#include<fstream>
using namespace ::std;
int main()
{
    char *filename = "BINARY";
    float height[3] = { 1.5, 2.5, 3.5 };
    ofstream outfile;
    outfile.open(filename);
    outfile.write((char *)&height, sizeof(height));
    outfile.close();
    for (int i = 0; i < 3; i++)
    {
        height[i] = 0;
    }
    ifstream infile;
    infile.open(filename);
    infile.read((char *)&height, sizeof(height));
    for (int i = 0; i < 3; i++)
    {
        cout << height[i] << endl;
    }
    infile.close();
    system("pause");
    return 0;
}
```

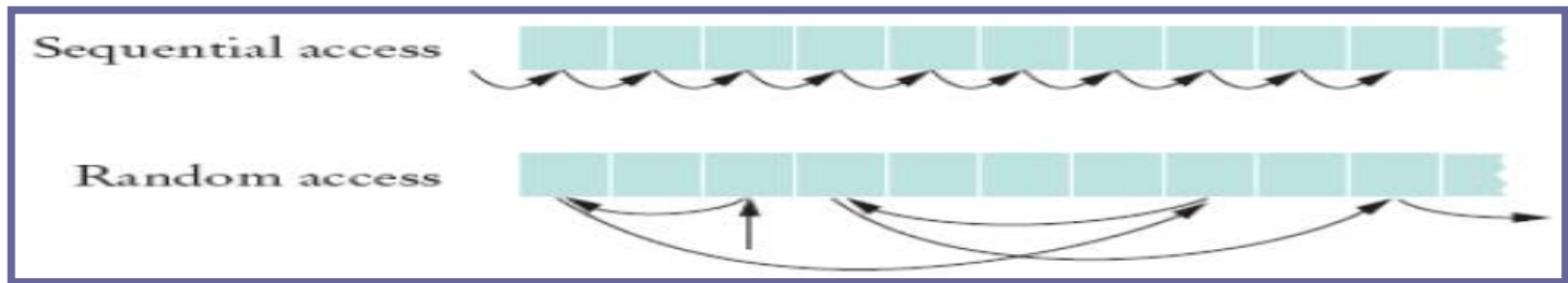
Writing File to a Specified Location

```
int main()
{
std::ofstream of("C:\\OOP\\log.txt");
of << "hello\n";
system("pause");
return 0;
• }
```

Random Access Files

Random access files are files in which records can be accessed in any order

1. Also called direct access files
2. More efficient than sequential access files



Need

1. Real-time applications require immediate response
 - a) Example: respond to customer query about a billSequencing through records for account is time-intensive
2. Random (immediate) access meets real-time need
 - a) Directly read from or write to desired record

Random Access Files

1. The Random Access File class contains the same read(), write() and close() methods as Input and Output Stream.
2. Also contains seek() that lets you select a beginning position within the file before reading or writing data.
3. Includes capabilities for reading and writing.
4. With a random-access file, you can seek to the desired position and then read and write an amount of bytes.

Writing an Object to the Disk

```
class person
{
protected:
    string name;
    int age;
public:
    void getData()
    {
        cout << "Name:" ;
        cin >> name;
        cout << "Age:";
        cin >> age;
    }
    void showData()
    {
        cout << "Name:" << name << endl;
        cout << "Age:" << age << endl;
    }
};

int main()
{
    person per;
    per.getData();
    ofstream outfile("PERSON", ios::binary);
    outfile.write((char *)&per, sizeof(per));
    outfile.close();
    ifstream infile("PERSON", ios::binary);
    infile.read((char *)&per, sizeof(per));
    per.showData();
    infile.close();
    system("pause");
    return 0;
}
```

I/O with Multiple Objects

```
#include<iostream>
#include<fstream>
#include<string>
using namespace ::std;
class person
{ private:
    string name;
    int age;
public:
    void getdata()
    {   cout << "enter name:";
        cin >> name;
        cout << "enter age:";
        cin >> age;
    }

    void showdata()
    {   cout << "name = " << name<<"\n";
        cout << "age = " << age<<"\n";
    }
};
```

```
int main()
{
    person per;
    char ch='y';
    fstream file;
    file.open("person.dat",ios::trunc|ios::in | ios::out | ios::binary);
    cout << "Enter person data:" << endl;
    cout << "Enter 'Y' or 'y' to enter data"<<endl;
    while (ch == 'Y' || ch == 'y')
    {
        per.getdata();
        file.write((char*)&per, sizeof(per));
        cin >> ch;
    }
    file.seekg(0);
    cout << "hi" << endl;
    while ((file.read((char *)&per, sizeof(per))))
    {
        per.showdata();
        
    }
    cout << endl;
    file.close();
    system("pause");
    return 0;
}
```

Example for seekg()

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;
```

```
class person
{
private:
    string name;
    int age;
public:
    void getData()
    {
        cout << "\n Enter Name:";
        cin >> name;
        cout << "Enter Age:";
        cin >> age;
    }
    void showData() { ... }
};
```

```
int main()
{
    person pers;
    ifstream infile;
    infile.open("GROUP.DAT", ios::in | ios::binary); //open file
    infile.seekg(0, ios::end); //go to 0 bytes from the end
    int endposition = infile.tellg(); //find where we are
    int n = endposition / sizeof(person); //number of persons
    cout << "\n There are " << n << "person in file";
    cout << "Enter person number:";
    cin >> n;
    int position = (n - 1)*sizeof(person); //number times size
    infile.seekg(position); //bytes from the start
    //read one person
    infile.read((char*)&pers, sizeof(pers));
    pers.showData(); //display the person
    cout << endl;
    infile.close();
    system("pause");
    return 0;
}
```

How to delete record from a file

To delete a record, following procedure is carried out :

1. Firstly, determine the position of the record to be deleted, by performing a search in the file.
2. Keep copying the records other than the record to be delete in a temporary file say temp.dat.
3. Do not copy the record to be deleted to temporary file, temp.dat. And Copy rest of the records to temp.dat.
4. Delete original file say oop2.dat as :

```
remove("oop2.dat");  
rename("temp.dat", "oop2.dat");
```

```

#include<iostream>
#include<fstream>
#include<string>
using namespace std;
/* ... */
class student
{
    int U_ID;
public:
    void getdata()
    {
        cout << "Roll:";
        cin >> U_ID;
    }
    void display()
    {
        cout << "Roll:" << U_ID << endl;
    }
    int U_ID1()
    {
        return U_ID;
    }
};

```

```

void writefile()
{
    student a;
    ofstream fout("student.dat", ios::binary);
    for (int i= 1; i <= 3; i++)
    {
        a.getdata();
        fout.write((char*)&a, sizeof(a));
    }
}

void readfile()
{
    student a;
    ifstream fin("student.dat", ios::binary);
    while (fin.read((char*)&a, sizeof(a)))
    {
        a.display();
    }
}

```

```
void deletefile(int r) //takes roll number that needs to be deleted
{
    student a;
    ofstream fout("temp.dat", ios::binary);
    ifstream fin("student.dat", ios::binary);
    while (fin.read((char*)&a, sizeof(a)))
        if (a.U_ID1() != r)
            fout.write((char*)&a, sizeof(a));
    fin.close();
    fout.close();
    remove("student.dat");
    rename("temp.dat", "student.dat");
}

void main()
{
    student a;
    int n;
    writefile();
    cout << "enter the U_ID. to be deleted:"; cin >> n;
    deletefile(n);
    cout << "Record Deleted\n";
    readfile();
    system("Pause");
}
```

Error Handling During File Operations

One of the following things may happen when dealing with files:

1. A file which we are attempting to open for reading does not exist.
2. We may attempt an invalid operation such as reading past the end- of -file.
3. There may not be any space in the disk for storing more data.
4. We may use an invalid file name.
5. We may attempt to perform an operation when the file is not opened for that purpose.

Error Handling Functions

FUNCTION	RETURN VALUE AND MEANING
<code>eof()</code>	returns true (non zero) if end of file is encountered while reading; otherwise return false(zero)
<code>fail()</code>	return true when an input or output operation has failed
<code>bad()</code>	returns true if an invalid operation is attempted or any unrecoverable error has occurred.
<code>good()</code>	returns true if no error has occurred.

These functions may be used in the appropriate places in a program to locate the status of a file stream and thereby to take the necessary corrective measures. Examples:

`Ifstream infile;`

`Infile.open("ABC");`

```
While(!infile.fail())  
{  
.....  
.....(process the file)  
.....  
}
```

```
If(infile.eof())  
{  
.....(terminate program normally)  
.....}  
  
Else if(infile.bad())  
{  
.....(report fatal error)  
}
```

```
/*int main()
{
    int i;
    while (true)
    {
        cout << "\n Enter an Integer:";
        cin >> i;
        if (cin.good())
        {
            cin.ignore(10, '\n');
            break;
        }
        cin.clear();
        cout << "Incorrect Input";
        cin.ignore(10, '\n');
    }
    cout << "integer is " << i;
    system("pause");
    return 0;
}*/
```

Some String Functions

- `String1.assign(string 2)`
- `String1.append(string2)`

Comparison

- `String1.compare(string2)`
- `String1.compare(s1,l1,string2,s2,l2)`
- `String1.compare(0,string2.length(),string2)`

Substring

`String1.substr(7,5)`

`String1.swap(string2)`